RESILIENCE REPORT

SaleSphere – Self-Healing CRM

Project Type: Self-Healing Software
Technology Stack: React, Node.js, Express.js and MongoDB

1. Introduction

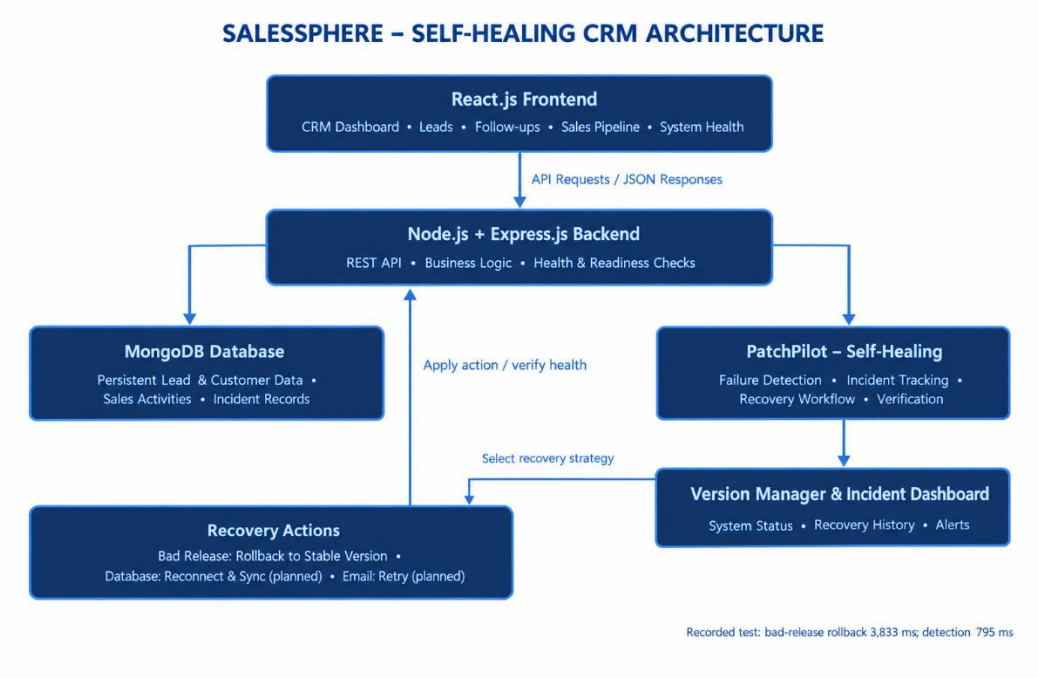
SaleSphere is a Customer Relationship Management (CRM) application designed to help manage customer leads, follow-ups, sales activities, and the sales pipeline. While developing this project, I focused not only on the normal working of the application but also on what happens when something goes wrong.

In a normal application, a failure may interrupt the user's work until someone identifies and fixes the problem manually. SaleSphere introduces a self-healing approach, its recovery component. The goal is to detect a failure, identify the affected component, attempt an appropriate recovery, and verify whether the application has returned to a healthy state.

The main objective is to reduce manual intervention and make failures visible through the System Health, Version Manager, and Incidents sections of the application.

2. System Architecture

SaleSphere follows a client-server architecture. The React frontend provides the interface, while the Node.js and Express.js backend handles API requests and application logic. MongoDB is used for persistent lead storage.



The diagram represents the intended recovery flow. The exact recovery action depends on the type of failure. For example, a bad software release can trigger rollback, while a database disconnection requires a fallback and synchronization strategy.

3. Failure Handling and Recovery

I selected failure scenarios that represent common problems in a CRM application. Each scenario needs a way to detect the issue and a suitable recovery action.

Failure	How It Is Detected	How It Heals	Measured Recovery Time
Bad software release	The controlled failure test introduces a faulty version, which is detected by the version-health and monitoring flow.	PatchPilot records the incident and restores the stable version through the rollback workflow.	**3,833 ms** in the recorded test; detection took **795 ms**.
MongoDB disconnection	The database connection state and readiness checks can indicate that MongoDB is unavailable.	The planned fallback uses cached lead data and queues changes locally for synchronization after reconnection. End-to-end sync must be verified.	Not yet confirmed by a recorded test.
Email delivery failure	A controlled failure flag activates the email recovery workflow when an email is sent.	PatchPilot records the incident, moves through recovery stages, and attempts email delivery. The final result depends on the mail service.	Not yet confirmed by a recorded test.
Backend/API unavailability	Health and readiness endpoints provide basic information about backend availability.	The failure can be exposed through health monitoring. Automatic restart of a crashed backend requires a separate, configured and tested supervisor.	Not measured.

The recovery time depends on the type of failure. Some failures can be resolved by restoring a previous software version, while others depend on external services or successful data synchronization.

4. Before-and-After Results

The clearest recorded result from the current project is the controlled bad-release rollback test.

Measurement	Recorded Result
Failure detection time	795 ms
Total recovery time	3,833 ms
Time to return to the stable version	Included in the recorded recovery time

Before self-healing: If a faulty release remains active, a developer may need to identify the issue and manually restore a stable version.

With the recovery workflow: The controlled test detected the faulty release, created an incident, moved through the recovery stages, and returned to the stable version. The recorded recovery took approximately 3.8 seconds.

For database failures, the intended improvement is to keep cached lead data available during a disconnection and synchronize queued changes after the database reconnects. For email failures, the recovery workflow makes the failure and retry status visible. These improvements should be reported as fully successful only after repeatable tests confirm them.

The current timing is based on one recorded test. It should not be treated as an average or a guarantee of the same performance in every environment.

5. Testing and Verification

To test the version-recovery workflow, I used a controlled failure scenario instead of waiting for a real production failure. The test introduced a faulty software version and allowed the recovery process to detect and handle it.

The verification process involved the following steps:

1. Trigger the controlled bad-release scenario.
2. Check whether the failure is detected and an incident is recorded.
3. Observe the recovery stages shown by PatchPilot.
4. Confirm that the stable version becomes active again.
5. Check that the incident status changes to recovered and review the recorded timings.

The same testing approach can be applied to the other scenarios. For database recovery, the important checks are whether leads remain available during disconnection and whether queued changes reach MongoDB after reconnection. For email recovery, the important checks are whether the retry succeeds and whether the final status matches the actual delivery result.

6. Known Limitations

SaleSphere is a student project developed in a controlled environment, so some limitations remain.

- **Limited performance measurements:** The current version-rollback result comes from one recorded test. Repeated tests are needed to calculate average or median recovery time.
- **Database consistency:** Local caching and queued operations must be tested for duplicate entries, conflicting updates, deletion handling, and successful synchronization after reconnection.
- **External email service:** Email recovery depends on valid credentials, network connectivity, and the availability of the email provider. A retry cannot guarantee delivery if the provider remains unavailable.
- **Backend process crashes:** Health endpoints can report service status, but they cannot independently restart a crashed Node.js process without an additional process supervisor.

- **Controlled failure testing:** A successful simulated rollback proves that the designed recovery path works in that test. It does not prove that every possible real-world failure will be detected or recovered automatically.
- **Production security:** Failure-injection endpoints should be restricted or disabled in production. Environment secrets must remain private and must not be committed to the source repository.

7. Conclusion

SaleSphere combines normal CRM functionality with a visible self-healing workflow supported by PatchPilot. The project demonstrates how failure detection, incident tracking, automatic version rollback, and recovery verification can reduce the need for manual intervention.

The recorded bad-release test completed recovery in 3,833 milliseconds, with failure detection taking 795 milliseconds. The next step is to repeat the tests and complete end-to-end verification of database reconnection, queued data synchronization, and email recovery.

This project helped me understand that self-healing software is not only about detecting an error. It must also take an appropriate recovery action and verify that the system has returned to a working state.